REGULAR EXPRESSIONS

The language accepted by DFA or NFA or ε-NFA is called Regular languages.

A regular language can be described using regular expressions consisting of the symbols such as alphabets in Σ, the operators such as '.', '+' and '*'.

+ operator - union operator

. operator - concatenation operator

* operator - closure operator

A regular expression is recursively defined as

* ϕ is a regular expression denoting an empty language
* ε is a regular expression indicates the language containing an empty string.
* a is a regular expression which indicates the language containing only 'a'.
* If R is a regular expression denoting the language L_R and S is a regular expression denoting the language L_S, then
  - R+S is a regular expression corresponding to the language $L_R \cup L_S$.
  - R.S is a regular expression corresponding to the language $L_R \cdot L_S$.
  - R^* is a regular expression corresponding to the language L_R^*.
* The expressions obtained by applying any of the above rules are regular expressions.

Examples :

Regular expressions	Meaning
a^*	String consisting of zero or more a's
a^+	String consisting of one or more a's
$(a+b)$ $(a \cup b)$	String consisting of either one a or one b
$(a+b)^*$ $(a \cup b)^*$	Set of strings of a's and b's of any length including the NULL string
$(a+b)^*abb$ $(a \cup b)^*abb$	Set of strings of a's and b's ending with the string abb
$ab(a+b)^*$ $ab(a \cup b)^*$	Set of strings of a's and b's starting with the string ab
$(a+b)^*aa(a+b)^*$ $(a \cup b)^*aa(a \cup b)^*$	Set of strings of a's and b's having a substring aa
$a^*b^*c^*$ $a^*b^+c^*$	Set of strings consisting of any number of a's (may be empty string also) followed by any number of b's (may include empty string) followed by any number of c's (may include empty string)
$a^+b^+c^+$	Set of string consisting of at least one 'a' followed by string consisting of at least one 'b' followed by string consisting of at least one 'c'.

$aa^*bb^*cc^*$	Set of strings consisting of at least one 'a' followed by string consisting of at least one 'b' followed by string consisting of at least one 'c'.
$(a+b)^*(a+bb)$	Set of strings of a's and b's ending with either a or bb.
$(aa)^*(bb)^*b$	Set of strings consisting of even number of a's followed by odd number of b's.
$(0+1)^*000$	Set of strings of 0's and 1's ending with three consecutive zeros or ending with 000
$(11)^*$	Set of strings consisting of even number of 1's
01^*+1	The language represented is the string 1 or the string consisting of a zero followed by any numbers of 1's possibly including none
$(01)^*+1$	The language consists of a string 1 or strings of (01)'s that repeat zero or more times
$0(1^*+1)$	Set of strings consisting of a zero followed by any numbers of 1's
$(1+\varepsilon)(00^*1)^*0^*$	Strings of 0's and 1's without any consecutive 1's
$(0+10)^*1^*$	Strings of 0's and 1's ending with any number of 1's (possibly none)
$(a+b)(a+b)$	Strings of a's and b's whose length is 2

Q1. Obtain a regular expression representing strings of a's and b's having length 2.

aa, ab, ba, bb

RE is $(aa+ab+ba+bb)$

$a(a+b)+b(a+b)$

OR

$(a+b)(a+b)$

RE is $(a+b)(a+b)$

Q2. Obtain a regular expression to accept strings of a's and b's of length ≤ 2.

RE :is $(\varepsilon+a+b)(\varepsilon+a+b)$

OR

RE :is $(\varepsilon+a+b)^2$

$\varepsilon + a + b + aa + ab + ba + bb$

$\varepsilon + a(\varepsilon + a + b) + b(\varepsilon + a + b)$

$(\varepsilon + a + b)(\varepsilon + a + b)$

Q3. Obtain a regular expression to accept strings of a's and b's of length ≤ 10.

R.E. :is $(\varepsilon + a+b)^{10}$

Q4. Obtain a regular expression representing strings of a's and b's having even length.

R.E. :is $(aa+ab+ba+bb)^*$

OR

R.E. :is $((a+b)(a+b))^*$

Q5. Obtain a regular expression representing strings of a's and b's having odd length.

R.E. :is $(aa+ab+ba+bb)^*(a+b)$

OR

$(a+b)(aa+ab+ba+bb)^*$

R.E. :is $((a+b)(a+b))^*(a+b)$

OR

$(a+b)((a+b)(a+b))^*$

Q6. Obtain a regular expression such that $L(R) = \{w \mid w \in \{0,1\}^*$ with at least three consecutive 0's.

R.E. :is $(0+1)^* 000 (0+1)^*$

$L(R) = \{(0+1)^m 000 (0+1)^n \mid m \geq 0 \text{ and } n \geq 0\}$

Q7. Obtain a regular expression to accept strings of 0's and 1's having no two consecutive zeros.

R.E. :is $(1+01)^*(0+\varepsilon)$

(OR) $(0+\varepsilon)(1+10)^*$

Q8. Obtain a regular expression to accept string of a's and b's starting with 'a' and ending with 'b'.

R.E. is $a(a+b)^*b$

Q9. Obtain a regular expression to accept string of a's and b's whose second symbol from the right end is a.

R.E. is $(a+b)^*a(a+b)$

Q10. Obtain a regular expression representing string of a's and b's whose tenth symbol from the right end is a.

R.E. is $(a+b)^*a(a+b)(a+b)(a+b)(a+b)(a+b)(a+b)(a+b)(a+b)(a+b)$

OR

R.E. is $(a+b)^*a(a+b)^9$

Q11. Obtain a regular expression to accept strings of a's and b's such that third symbol from the right is a and fourth symbol from the right is b.

R.E. $(a+b)^*ba(a+b)(a+b)$

Q12. Obtain a regular expression to accept the words with two or more letters but beginning and ending with the same letter where $\Sigma = \{a, b\}$

R.E. is $a(a+b)^*a + b(a+b)^*b$

Q13. Obtain a regular expression to accept strings of a's and b's whose length is either even or multiples of 3 or both.

R.E. is $((a+b)(a+b))^* + ((a+b)(a+b)(a+b))^*$

Q14. Obtain a regular expression to accept strings of a's and b's such that every block of four consecutive symbols contains at least two a's.

R.E. is $(aa(a+b)(a+b) + a(a+b)a(a+b) + a(a+b)(a+b)a + (a+b)aa(a+b) + (a+b)a(a+b)a + (a+b)(a+b)aa)^+$

Q15. Obtain a regular expression for the language $L = \{a^n b^m \mid m+n \text{ is even}\}$

R.E. $= (aa)^*(bb)^* + a(aa)^* b(bb)^*$

Even followed by even
$(m+n)$ is even $(aa)^*(bb)^*$
odd followed by odd $\equiv$ even
$(m+n)$ is even $a(aa)^* b(bb)^*$

even + even = even
odd + odd = even

Q16. Obtain a regular expression for the language $L = \{a^n b^m \mid m \geq 1, n \geq 1, nm \geq 3\}$

R.E. $= aaaa^*b + abbbb^* + aaa^*bbb^*$

if $m = 1$, $n \geq 3$ $aaaa^*b$ $nm \geq 3$

if $n = 1$, $m \geq 3$ $abbbb^*$ $nm \geq 3$

if $nm \geq 3$, if $m \geq 2$ then $n \geq 2$ aaa^*bbb^*

Q17. Obtain a regular expression for the language $L = \{a^{2n} b^{2m} \mid n \geq 0, m \geq 0\}$

R.E. $= (aa)^*(bb)^*$

if $n \geq 0$, a^{2n} even no of a's $(aa)^*$

$m \geq 0$, b^{2m} even no of b's $(bb)^*$

Q18. Obtain a regular expression for strings of a's and b's containing not more than three a's.

R.E. is $b^*(\varepsilon+a)b^*(\varepsilon+a)b^*(\varepsilon+a)b^*$

ε, a, aa, aaa $\}$ $(\varepsilon+a)(\varepsilon+a)(\varepsilon+a)$

Q19. Obtain a regular expression for the language $L = \{a^n b^m : n \geq 4, m \leq 3\}$

R.E. is $aaaa(a)^*(\varepsilon+b)(\varepsilon+b)(\varepsilon+b)$

a^n, $n \geq 4$, $aaaa(a)^*$

$\varepsilon/b/bb/bbb$

Q20. Obtain a regular expression for strings of a's and b's whose lengths are multiples of 3.

R.E. is $((a+b)(a+b)(a+b))^*$

obtain a RE to accept strings of 0's and 1's having no two consecutive zero's.

$(1+01)^*(0+\varepsilon)$ or $(0+\varepsilon)(1+10)^*$

Name of Experiment..... Date..... Page No. 211

Experiment No..... Experiment Result.....

Q21. Obtain a regular expression for the language $L=\{w : |w| \bmod 3 = 0$ where $w \in \{a,b)^*\}$.

R.E. is $((a+b)(a+b)(a+b))^*$

Q22. Obtain a regular expression for the language

$L=\{w : n_a(w) \bmod 3 = 0$ where $w \in (a,b)^*\}$

R.E. is $(b^*ab^*ab^*ab^*)^* b^*$

Q23. Obtain a regular expression for the set of all strings that do not end with 01, over $\{0,1\}^*$.

R.E. is $(0+1)^*(00+10+11)$

Q24. Obtain a regular expression for $L=\{vuv : u,v \in \{a,b\}^*$ and $|v|=2\}$

R.E. is $aa(a+b)^*aa + ab(a+b)^*ab + ba(a+b)^*ba + bb(a+b)^*bb$

Q25. Obtain a regular expression for $L=\{w$: string ends with ab or ba where $w \in \{a,b\}^*\}$.

R.E. is $(a+b)^*(ab+ba)$

NOTE: Precedence of regular expression operators

<1> The star operator (*) has the highest precedence

<2> The concatenation operator / dot operator has the next highest precedence

<3> The union operator (+) has the least precedence.

Ex: $(a+b)a^*$

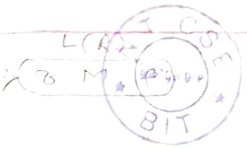
<1> $(a+b)$

<2> $*$

<3> $\cdot$ <4> $+$

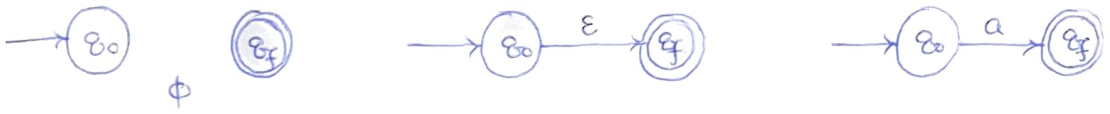
The schematic representation of a regular expression R to accept the language L(R) is shown as where q_0 is the start state and q_f is the final state of the machine M.

Obtain ε-NFA from Regular Expression



Q. Prove that there exists a finite automaton to accept the language L(R) corresponding to the regular expression R.

By definition, ϕ, ε, and a are regular expressions.



Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_1, f_1)$ be a machine which accepts the language $L(R_1)$ corresponding to the regular expression R_1.

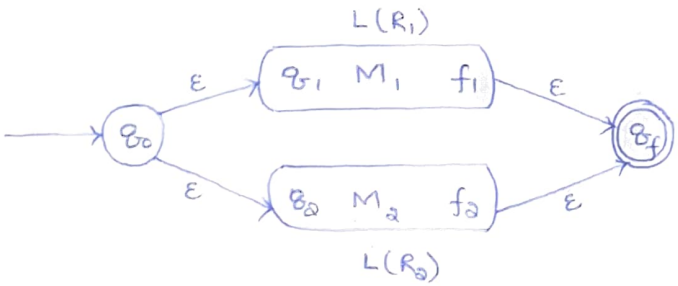
Let $M_2 = (Q_2, \Sigma_2, \delta_2, q_2, f_2)$ be a machine which accepts the language $L(R_2)$ corresponding to the regular expression R_2.

The various machines corresponding to the regular expressions $R_1 + R_2$, $R_1 \cdot R_2$ and R^* are shown below:

Case 1 :

$R = R_1 + R_2$

An NFA which accepts either $L(R_1)$ or $L(R_2)$ which can be represented as $L(R_1 + R_2)$ as shown below

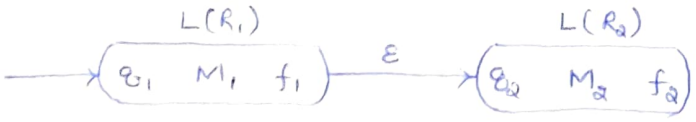


q_0 is the start state of the combined machine and q_f is the final state of combined machine M.

Case 2 :

$R = R_1 \cdot R_2$

An NFA which accepts $L(R_1)$ followed by $L(R_2)$ which can be represented as $L(R_1 \cdot R_2)$ as shown below



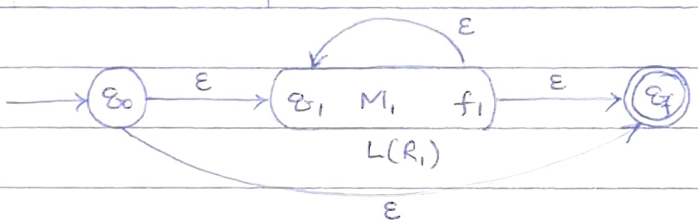
The machine after accepting $L(R_1)$ moves from state q_1 to f_1. Since there is a ε-transition, without any input there will be a transition from state f_1 to state q_2.

In state q_0, upon accepting $L(R_2)$, the machine moves to f_2 which is the final state. Thus, q_1 which is the start state of machine M_1 becomes the start state of the combined machine M and f_2 which is the final state of machine M_2, becomes the final state of machine M and accepts the language $L(R_1 \cdot R_2)$.

Case 3 :

$R = (R_1)^*$

An NFA which accepts $L(R_1)^*$ is as shown below

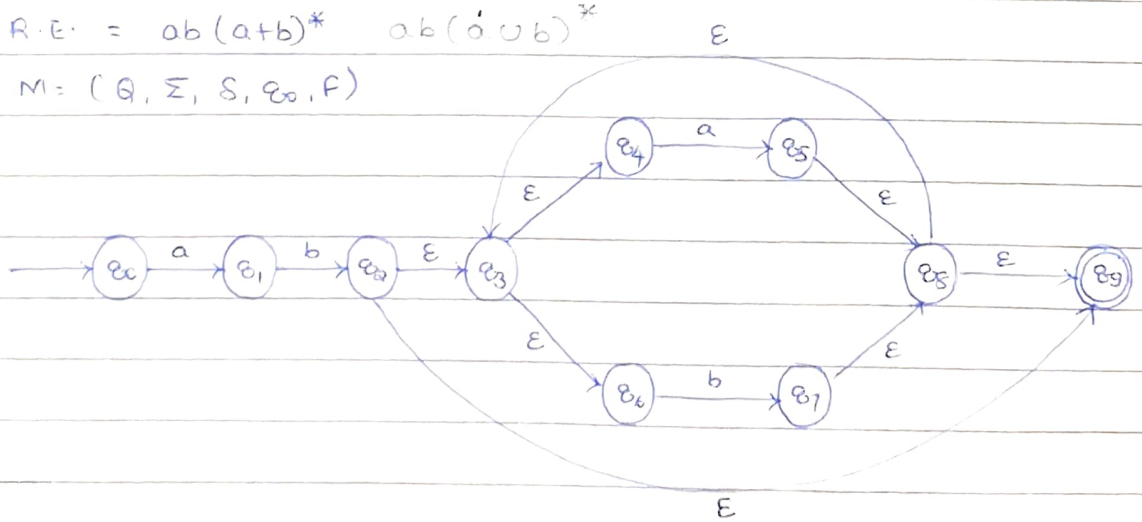


The machine can either accept ε or any number of $L(R_1)$'s thus accepting the language $L(R_1)^*$. Here, q_0 is the start state & q_f is the final state.

Q1. Obtain an ε-NFA which accepts strings of a's and b's starting with the string ab.

R.E. = $ab(a+b)^*$ $ab(a \cup b)^*$

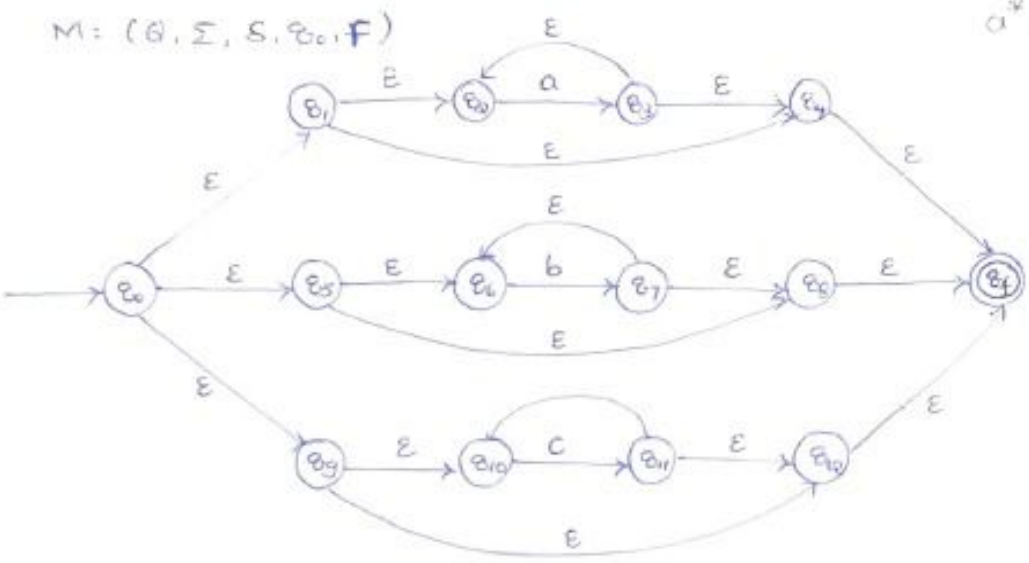
M = $(Q, \Sigma, \delta, q_0, F)$



Q2. Obtain ε-NFA for the regular expression $a^* + b^* + c^*$.

$M: (Q, \Sigma, \delta, q_0, F)$

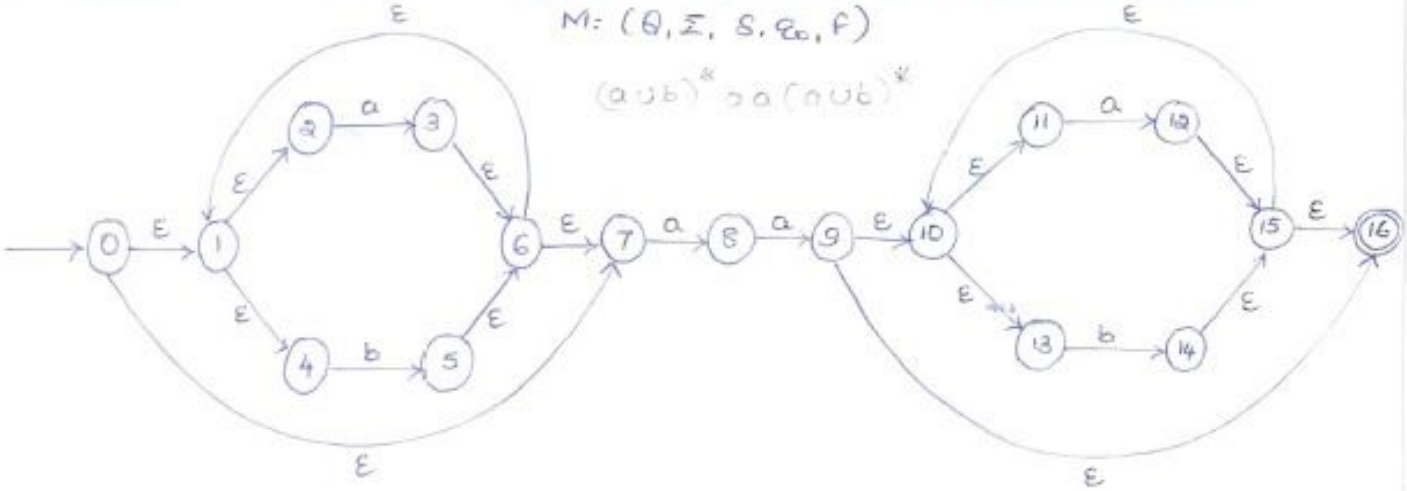
$a^* \cup b^* \cup c^*$



Q3. Obtain NFA for the regular expression $(a+b)^* aa (a+b)^*$

$M: (Q, \Sigma, \delta, q_0, F)$

$(a \cup b)^* aa (a \cup b)^*$

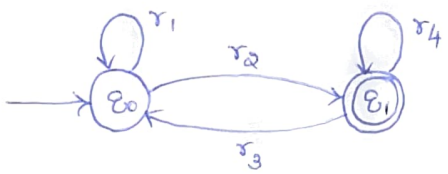


Obtain RE from FA (By Eliminating States)

Theorem :

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a FA recognizing the language L. Then there exists an equivalent regular expression R for the regular language L such that $L = L(R)$.

The general procedure to obtain a R.E. from FA is shown below :



Consider the generalized graph, where r_1, r_2, r_3 & r_4 are the regular expressions.

So, $r = r_1^* r_2 (r_4 + r_3 r_1^* r_2)^*$

NOTE :

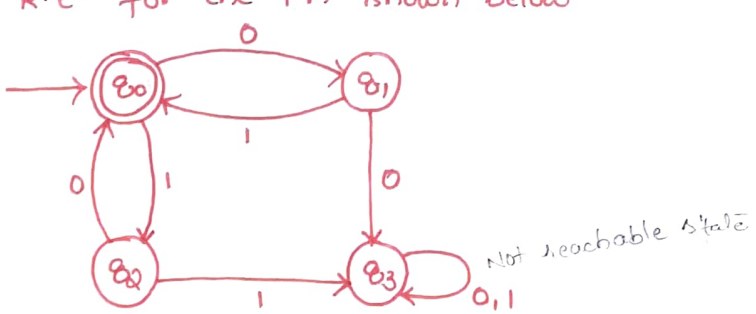
<1> If r_3 is not there, then

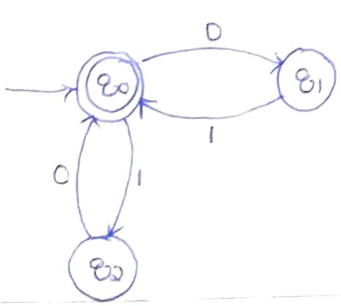
$r = r_1^* r_2 r_4^*$

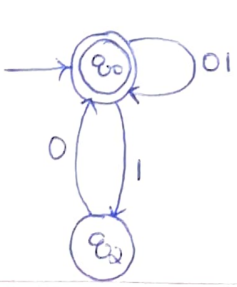
<2> If q_0 and q_1 are the final states without r_3, then

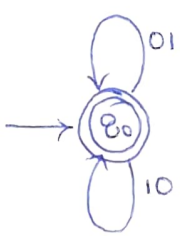
$r = r_1^* + r_1^* r_2 r_4^*$

Q1. Obtain a R.E. for the FA shown below



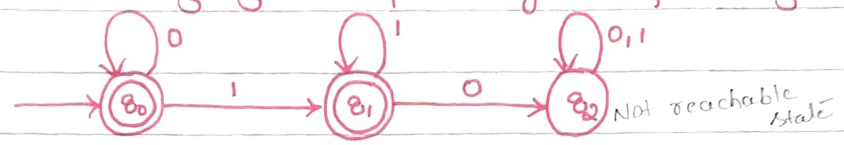


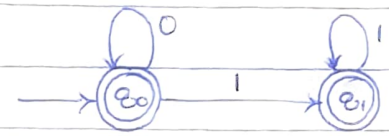




R.E. $= (01 + 10)^*$

Q2. What is the language accepted by the following FA.





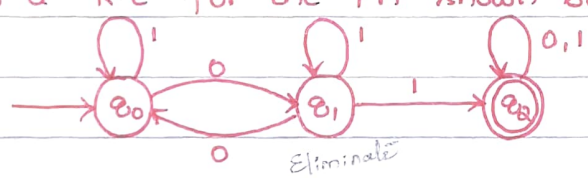
R.E = $0^* + 0^*11^*$

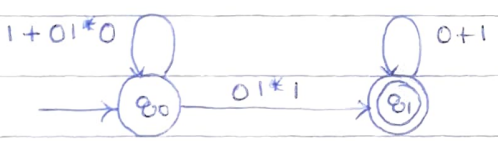
$= 0^*(\varepsilon + 11^*)$

$= 0^*1^*$

Language consists of any number of 0's followed by any number of 1's.

Q3. Obtain a R.E. for the FA shown below





$r_1 = 1 + 01^*0$

$r_2 = 01^*1$

$r_4 = 0 + 1$

R.E. $r = r_1^* r_2 r_4^*$

$r = (1 + 01^*0)^* \; 01^*1 \; (0+1)^*$

Applications of Regular Expressions

* Regular expressions in UNIX

→ Regular expressions are extensively used in UNIX operating system. Ex: the symbol '.' stands for any character, the sequence [abcde...] stands for the regular expression "a+b+c+d+e...", the operator | is used in place of +, the operator ? means "zero or one of" etc.

→ Most of the commands are invoked invariably uses RE.

for example : grep (Global search for Regular Expression and Print) used to search for a pattern of string.

* Pattern Matching

We can match an identifier or a decimal number or we can search for a string in the text.

* Lexical Analysis

→ RE are extensively used in the design of lexical analyzer phase. This is the first phase of the compiler design. This phase scans the source program and recognizes all the tokens which are logically together.

→ The UNIX commands such as lex accepts regular expressions as the input and produces the lexical analyzer generator. This generator takes a high-level description of a lexical analyzer as the input and produces lexical analyzer.

* Obtain a regular expression to identify an identifier.

An identifiers starts with a letter. This letter can be followed by combination of zero or more letters and digits i.e., an identifier can be a single letter followed by strings of letters and digits of any length and can be represented as

$$\text{letter}\ (\text{letter} + \text{digit})^*$$

* Obtain a RE to identify an integer

An integer can start with any of the signs +, − or ε followed by one or more digits ranging from 0 to 9. This can be represented using a RE as

sd^+ or sdd^*

where s stands for sign and d stands for the digits from 0 to 9.

* An application of RE in UNIX editor ed.

In UNIX operating system, we can use the editor ed to search for a specific pattern in the text.

for example : if the command specified is /ab*c/ then the editor searches for a string which starts with a followed by zero or more b's and followed by the symbol c

The editor ed accepts the RE and searches for that particular pattern in the text.

[redacted]

REGULAR LANGUAGES

PROPERTIES OF REGULAR LANGUAGES

The regular languages are the languages accepted by DFA's, NFA's and ε-NFA's and defined by regular expressions.

The two important properties of regular languages are

* closure property of regular languages (union & intersection)
  - This property is a useful tool for building complex automata.
  - Using the closure property we can build language recognizers.
  - for example: the intersection of two regular languages is also regular. So, if there are two automata recognizing two different languages, we can construct an automaton that recognizes the intersection of these two languages.

* Decision property of regular languages (2 FA's accept the same language or not)
  - Using this property, we can decide whether two automata define the same language. If so, we can minimize the states of automata with as few states as possible. The minimization of automaton is very important in the design of switching circuits. This is because, as the number of states of automaton decreases the size of the circuit and hence the cost decreases.

Any finite language can be expressed using regular expressions and we can construct finite automata (DFA or NFA or ε-NFA). Though the regular languages are important, there are non-regular languages which are very interesting and important.

for example:
* $L = \{ w : w \in \{0,1\}^* \text{ and has equal number of 0's and 1's} \}$
* $L = \{ 0^n 1^n \text{ for } n \geq 0 \}$
* $L = \{ a^p \in \text{ where } p \geq 2 \text{ is a prime number} \}$
* Language consisting of matching parentheses

We can prove that certain languages are not regular using one powerful tool called pumping lemma.

PUMPING LEMMA FOR REGULAR LANGUAGES

"State and prove pumping lemma for regular languages".

Theorem

Let $M = (K, \Sigma, \delta, s, A)$ (FSM) $(Q, \Sigma, \delta, q_0, F)$ be an ~~FA~~ and has n number of states. Let L be the regular language accepted by M. Let for every string x in L, there exists a constant n such that $|x| \geq n$. If the string x can be broken into three substrings u, v and w such that

$$x = uvw$$

satisfying the following constraints:

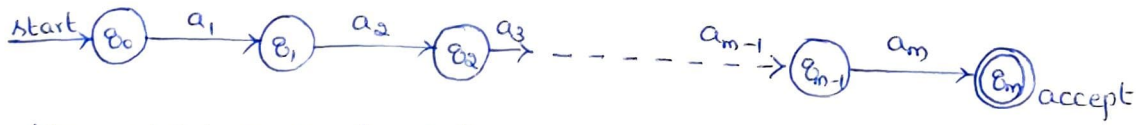
- $v \neq \varepsilon$ i.e., $|v| \geq 1$
- $|uv| \leq n$

then $uv^i w$ is in L for $i \geq 0$.

Proof

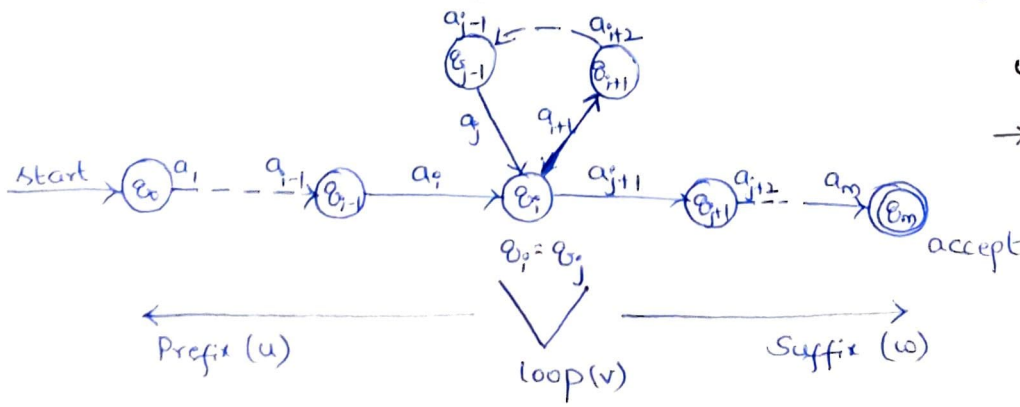
Let $M = (K, \Sigma, \delta, s, A)$ (FSM) $(Q, \Sigma, \delta, q_0, F)$ be an FA (FSM) and let L is the language accepted by ~~DFA~~ (FSM) and is regular. Let $x = a_1 a_2 a_3 \cdots a_m$ where $m \geq n$ and each a_i is in Σ. Here, n represent the states of DFA (FSM).

Since we have m input symbols, we should have m+1 states in the sequence $q_0, q_1, q_2, \cdots, q_m$, where q_0 will be the start state and q_m will be the final state as shown below.



Let the string x is divided into three substrings as shown below:

- the string prefix from $a_1, a_2, \cdots a_i$ i.e., $u = a_1 a_2 \cdots a_i$
- the loop string from $a_{i+1} a_{i+2} \cdots a_{j-1} a_j$ i.e., $v = a_{i+1} a_{i+2} \cdots a_{j-1} a_j$
- the string suffix from $a_{j+1} a_{j+2} \cdots a_m$ i.e., $w = a_{j+1} a_{j+2} \cdots a_m$



The prefix string u takes the machine from q_0 to q_i, the loop string v takes the machine from q_i to q_j and suffix string w takes the machine from q_j to q_m.

The minimum string that can be accepted by the above ~~FA~~ FSM is uw with $i = 0$.

But when $i = 1$, the string uvw is accepted by ~~DFA~~ FSM.

when $i = 2$, the string uvvw is accepted by ~~DFA~~ FSM.

So, if $i > 0$, the machine goes from q_0 to q_i on input string u, circles from q_i to q_j based on the value of i and then goes to accepting state on input string w.

In general, if the string x is split into sub strings uvw, then for all $i \geq 0$, $uv^iw \in L$.

This can be expressed using the transitions as shown below

$$\delta(q_0, a_1 a_2 \cdots a_{i-1} a_i a_{j+1} a_{j+2} \cdots a_m) = q_m$$

$$= \delta(\delta(q_0, a_1 a_2 \cdots a_{i-1} a_i), a_{j+1} a_{j+2} \cdots a_m)$$

$$= \delta(q_i, a_{j+1} a_{j+2} \cdots a_m)$$

$$= q_m$$

After the string $a_1 a_2 \cdots a_i$ the machine will be in state q_i. Since q_i and q_j are same, we can input the string $a_{i+1} a_{i+2} \cdots a_j$ any number of times and the machine will stay in q_j only. finally, if the input string is $a_{j+1} a_{j+2} \cdots a_m$ the machine enters into final state q_m.

Hence, the proof.

APPLICATIONS OF PUMPING LEMMA

* Pumping Lemma is used to prove that certain languages are non-regular. But, it cannot be used to prove that a given language is regular.
* Using pumping lemma, it is possible to check whether a language accepted by ~~FA~~ FSM is finite or infinite.

The general strategy used to prove that certain language are not regular is as shown below:

STEP 1 : Assume that the language L is regular and n be the number of states of FA. FSM

STEP 2 : Select the string x such that $|x| \geq n$ and break it into substrings u, v and w so that $x = uvw$ with the constraints:

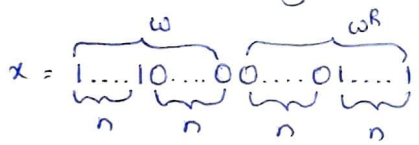
- $v \neq \varepsilon$ i.e., $|v| \geq 1$
- $|uv| \leq n$

STEP 3 : Find any i such that uv^iw is not in L i.e., $uv^iw \notin L$. According to pumping lemma, uv^iw is in L for $i \geq 0$. So, the result is contradiction to the assumption that the language is regular. Therefore, the given language L is not regular.

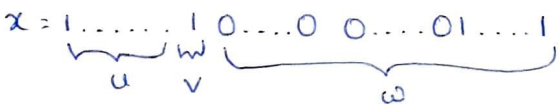
Q1. Show that $L = \{ww^R \mid w \in (0+1)^*\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states in FA FSM.

Consider the string



STEP 2 : Since $|x| \geq n$, we can split the string x into uvw such that $|uv| \leq n$ and $|v| \geq 1$ as shown below:



where $|u| = n-1$ and $|v| = 1$ so that $|uv| = |u| + |v| = n+1-1 = n$ which is true. According to pumping lemma, $uv^iw \in L$ for $i = 0, 1, 2, \ldots\ldots$

STEP 3 : If $i = 0$ i.e., v does not appear and so the number of 1's on the left of x will be less than the number of 1's on the right of x and so the string is not of the form ww^R. So, $uv^iw \notin L$ when $i = 0$. This is a contradiction to the assumption that the language is regular.

So, the Language $L = \{ww^R \mid w \in (0+1)^*\}$ is not regular.

Q2. Show that $L = \{a^n b^n \mid n \geq 0\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states in ~~FA~~ FSM

Consider the string $x = a^n b^n$

STEP 2 : Since $|x| = 2n \geq n$, we can split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$ as shown below

$x = \overbrace{aaaaaa}^{n}\ \underbrace{a}_{v}\ \overbrace{bbbbbb}^{n}$ (u, v, w)

where $|u| = n-1$ and $|v| = 1$ so that $|uv| = |u| + |v| = n-1+1 = n$ and $|w| = n$. According to pumping lemma, $uv^i w \in L$ for $i = 0, 1, 2, \ldots$.

STEP 3 : If $i = 0$, the string v does not appear and so the number of a's will be less than the numbers of b's and so the string x does not have n number of a's followed by n number of b's. But, according to pumping lemma, n number of a's should be followed by n number of b's which is a ~~contradiction~~ contradiction to the assumption that the language is regular.

So, the Language $L = \{a^n b^n \mid n \geq 0\}$ is not regular.

Q3. Show that $L = \{a^i b^j \mid i > j\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states ~~of~~ ~~FA~~ in FSM.

consider the string $x = a^{n+1} b^n$

STEP 2 : Since $|x| = 2n+1 \geq n$, we can split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$ as shown below

$x = a^{n+1} b^n = \underbrace{a^j}_{u}\ \underbrace{a^k}_{v}\ \underbrace{ab^n}_{w}$

where $|u| = j$ and $|v| = k \geq 1$ and so that $|uv| = |u| + |v| = j + k \leq n$

STEP 3 : According to pumping lemma, $uv^i w \in L$ for $i \geq 0$.

i.e., $a^j (a^k)^i ab^n \in L$ for $i \geq 0$.

if $i = 0$, number of a's in string u will not be more than the number of b's in w which is contradiction to the assumption that number of a's are more than the number of b's.

So, the Language $L = \{a^i b^j \mid i > j\}$ is not regular.

Q4. Show that $L = \{a^n b^i \mid n \neq 1\}$ is not regular.

Solution is same as Q3.

$x = a\ a\ b$
$\downarrow\ \downarrow\ \downarrow$
$u\ v\ w$

Q5. Show that $L = \{a^n b^1 c^{n+1} \mid n+1 \geq 0\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states in FA.

STEP 2 : Since L is regular, it is closed under homomorphism.

So, we can take

map btw two algebraic structures of the same type.

$h(a) = a \quad h(b) = a \quad \text{and} \quad h(c) = c$

the Language L is reduced to

$L = \{a^n a^1 c^{n+1} \mid n+1 \geq 0\}$

OR

$L = \{a^{n+1} c^{n+1} \mid n+1 \geq 0\}$

OR

$L = \{a^i b^i \mid i \geq 0\}$

Above Language is not regular, which is contradiction to the assumption that the language is regular. So, the given Language $L = \{a^n b^1 c^{n+1} \mid n+1 \geq 0\}$ is not regular.

Q6. Show that $L = \{a^{n!} \mid n \geq 0\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states in FA.

STEP 2 : Let $x = a^{n!}$ where $|x| \geq n$.

So, we can split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$ as shown below

$x = a^{n!} = a^j \; a^k \; a^{n!-j-k}$

$u \quad v \quad w$

$x = a^{3!} \quad 3*2*1$

$a^j \; a^k \; a^{n!-j-k}$

$j=3 \quad k=2 \quad n!-j-k=1 \quad (3*2*1-3-2$

$6-3-2$

$3-2$

$1)$

where $|u| = j$ and $|v| = k \geq 1$ and so that $|uv| = |u| + |v| = j+k \leq n$.

STEP 3 : According to pumping lemma, $uv^i w \in L$ for $i = 0, 1, 2, \ldots$

i.e., $a^j (a^k)^i a^{n!-j-k} \in L$

if $i = 0$, $a^j a^{n!-j-k} \in L$ i.e., $uw \in L$

$a^{n!-k} \in L$

So, $n! > n! - k$

$3*2*1 > 3*2*1 - 2*1$

$6 > 6-2$

$6 > 4$

$6 > 6-1$

$6 > 5$

when $k = 1$, $n! > n! - 1$

But according to Pumping Lemma $n! = n! - 1$ which is not and is a contradiction. So, L cannot be regular.

So, the language $L = \{a^{n!} \mid n \geq 0\}$ is not regular.

$0! = 1 \quad 1 > 0$

$1 > 1 - 1$

Q7. Show that $L = \{0^n \mid n \text{ is prime}\}$ is not regular.

The Language generated from this is,

$L = \{00, 000, 00000, \ldots\}$

STEP 1 : Let L is regular and n be the number of states in FA.

Let $x = 0^n \in L$ where n is prime.

STEP 2: $|x| = n$, split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$.

$x = 0^n = 0^j \; 0^k \; 0^{n-j-k}$

$u \quad v \quad w$

$n = 7$

000 00 00

u v w

j k n-j-k

where $|u| = j$ and $|v| = k \geq 1$, so that $|uv| = |u| + |v| = j + k \leq n$.

STEP 3: According to pumping lemma, $uv^iw \in L$ for $i = 0, 1, 2, \ldots$.

$0^j \; (0^k)^i \; 0^{n-j-k} \in L$

$j + ki + n - j - k = n + ki - k$

$= n + k(i-1)$ is prime for all $i \geq 0$.

if $i = n+1$ then, $n + k(n+1-1)$

$= n + kn$

$= n(k+1)$ is also prime for each $k \geq 1$

which is contradiction, because if $k = 1$, it will not be a prime, to the assumption that the Language is regular. So, the Language $L = \{0^n \mid n \text{ is prime}\}$ is not regular.

Q8. Show that $L = \{w \mid n_a(w) = n_b(w)\}$ is not regular. Is L^* regular?

The language generated from this is,

$L = \{\varepsilon, ab, ba, abab, bbaa, \ldots\}$

STEP 1 : Let L is regular and n be the numbers of states in FA.

Let $x = a^n b^n \in L$

STEP 2: $|x| = 2n \geq n$, split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$.

$x = a^n b^n = a^j \; a^k \; b^n$

$u \quad v \quad w$

where $|u| = j$, $|v| = k \geq 1$, so that $|uv| = |u| + |v| = j + k \leq n$.

STEP 3: According to pumping lemma, $uv^iw \in L$ for $i = 0, 1, 2, \ldots$.

$a^j (a^k)^i \; b^n \in L$ for $i = 0, 1, 2, \ldots$

if $i = 0$, number of a's will be less than the number of b's and if $i = 2$, uv^2w will have more a's than b's which is contradiction to the assumption that it has equal number of a's and b's.

So, the language $L = \{w \mid n_a(w) = n_b(w)\}$ is not regular. Since L is not regular L^* is also not regular.

Q9. Show that $L = \{w \mid n_a(w) < n_b(w)\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states of FA.

Let $x = a^{n-1}b^n \in L$

STEP 2 : $|x| = 2n-1 \geq n$, split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$

$$x = a^{n-1}b^n = a^{n-1}\ b^k\ b^{n-k}$$
$$u\quad v\quad w$$

where $|u| = n-1$, $|v| = k$, so that $|uv| = |u|+|v| = n-1+k \leq n$.

To satisfy this condition the value of k should be less than or equal to 1.

STEP 3 : According to pumping lemma, $uv^iw \in L$ for $i = 0, 1, 2, \ldots$

$a^{n-1}(b^k)^i\ b^{n-k} \in L$ for $i = 0, 1, 2, \ldots$

if $i = 0$, $a^{n-1}b^{n-k} \in L$

$\Downarrow$

$a^{n-1}b^{n-1} \in L$ when $k = 1$

when $i = 0$, the number of a's and b's are same, which is contradiction to the assumption that the number of a's will be less than the number of b's.

So, the language $L = \{w \mid n_a(w) < n_b(w)\}$ is not regular.

Q10. Show that $L = \{ww \mid w \in \{a,b\}^*\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states in FA.

Let $x = a^nba^nb \in L$

STEP 2 : $|x| = 2n+2 \geq n$, split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$.

$$x = a^nba^nb = a^j\ a^k\ ba^nb$$
$$u\quad v\quad w$$

where $|u| = j$, $|v| = k \geq 1$, so that $|uv| = |u|+|v| = j+k \leq n$

STEP 3 : According to pumping lemma, $uv^iw \in L$ for $i = 0, 1, 2, \ldots$.

$a^j(a^k)^i\ ba^nb \in L$ for $i = 0, 1, 2, \ldots$

if $i = 0$, number of a's on the left of first b will be less than the number of a's after the first b which is contradiction to the assumption that they are not equal.

So, the language $L = \{ww \mid w \in \{a,b\}^*\}$ is not regular.

Q11. Show that $L = \{ a^n \mid n = k^2 \text{ for } k \geq 0 \}$ is not regular.

OR

Show that $L = \{a^n \mid n \text{ is a perfect square}\}$ is not regular.

STEP 1 : Let L is regular and n be the number of states in FA.

Let $x = a^m \in L$ where $m = n^2$.

STEP 2 : $|x| \geq n$, split x into uvw such that $|uv| \leq n$ and $|v| \geq 1$.

$x = a^m = a^j \; a^k \; a^{m-j-k}$

$u \quad v \quad w$

where $|u| = j$ and $|v| = k \geq 1$, so that $|uv| = |u| + |v| = j + k \leq n$.

STEP 3 : According to pumping lemma, $uv^iw \in L$ for $i = 0, 1, 2, \ldots$.

$a^j \; a^{ki} \; a^{m-j-k} \in L$ for $i = 0, 1, 2, \ldots$.

if $i = 2$, $uv^2w \in L \quad a^j a^{2k} a^{m-j-k} \in L \Rightarrow a^{k+m} \in L$

Since, $k \geq 1$, $|a^{m+k}| = m + k = n^2 + k$ (Since $m = n^2$)

$n^2 < n^2 + k < n^2 + 1$ (when $k = 1$) $< (n^2 + 2n + 1) = (n+1)^2$

So, $n^2 < (n+1)^2$

Since $n^2 + k$ $(m+k)$ lies between $n^2 + (n+1)^2$, it is not a perfect square which is contradiction to the assumption that it should be a perfect square, according to pumping lemma.

So, the language $L = \{ a^n \mid n = k^2 \text{ for } k \geq 0 \}$ is not regular.

Q12. Prove that the following languages are not regular.

i. $L = \{ 0^n 1^m 0^n \mid n, m >= 1 \}$

ii. $L = \{ 0^n 1^{2n} \mid n >= 1 \}$

iii. $L = \{ a^n b^{n+1} \mid n > 1 \}$

iv. $L = \{ 0^n \mid 0^n \mid n \geq 1 \}$

v. $L = \{ 0^n 1^m 0^n \mid n \text{ and } m \text{ are arbitrary integers} \}$

vi. $L = \{ 0^n 1^m \mid n \leq m \}$

vii. $L = \{ 0^n 1^{2n} \mid n \geq 1 \}$

viii. $L = \{ 0^n \mid n \text{ is perfect square} \}$

ix. $L = \{ 0^n \mid n \text{ is a perfect cube} \}$

x. $L = \{ 0^n \mid n \text{ is a power of } 2 \}$

PROPERTIES OF REGULAR LANGUAGES

Some new languages can be constructed from already existing languages using certain operations such as union, intersection, concatenation etc. We can build recognizers for these new languages using some of the properties of regular languages. These properties are called closure properties.

for example : the intersection of two regular languages is regular. Thus, given two finite automata that recognized two different languages, we can construct an automaton that recognizes the intersection of these two languages. The automaton thus obtained may have more states than either of the two automata. Thus using the closure properties we can build more complex automata.

The closure properties of regular languages are shown below:

* Union of two RLs is regular
* Intersection of two RLs is regular
* Complement of a RL is regular

} Boolean operations

* Closure (star) of a RL is regular
* Concatenation of two RLs is regular
* Difference of two RLs is regular
* Reversal of a RL is regular
* Homomorphism of a RL is regular
* Inverse homomorphism of a RL is regular

REGULAR LANGUAGES ARE CLOSED UNDER UNION, CONCATENATION AND STAR

"Show that if L_1 and L_2 are regular, then $L_1 \cup L_2$, $L_1 \cdot L_2$ & L_1^* are also regular".

Theorem

If L_1 and L_2 are regular, then $L_1 \cup L_2$, $L_1 \cdot L_2$ and L_1^* also denote the regular languages.

Proof

It is given that L_1 and L_2 are regular languages. So, there exists regular expressions R1 and R2 such that : $L1 = L(R1)$

$L2 = L(R2)$

By the definition of regular expressions,

- $R1 + R2$ is a regular expression denoting the language $L1 \cup L2$
- $R1 \cdot R2$ is a regular expression denoting the language $L1 \cdot L2$
- $R1^*$ is a regular expression denoting the language $L1^*$

So, the regular languages are closed under union, concatenation and star operations.

CLOSURE UNDER COMPLEMENTATION

"Show that if L is a regular language, then complement of L denoted by $\bar{L}$ is also regular".

Theorem

If L is regular language, then the complement of L denoted by $\bar{L}$ is also regular. (The set of regular languages is closed under complementation.)

Proof

Let $M1 = (Q, \Sigma, \delta, q_0, F)$ (K, Σ, δ, s, A) be a DFA which accepts the language L. Since, the language is accepted by a DFA, the language is regular. Let us define the machine $M2 = (Q, \Sigma, \delta, q_0, F)$ (K, Σ, δ, s, A) which accepts $\bar{L}$.

There is no difference between M1 and M2 except the final states. The non-final states of M1 are the final states of M2 and final states of M1 are the non-final states of M2.

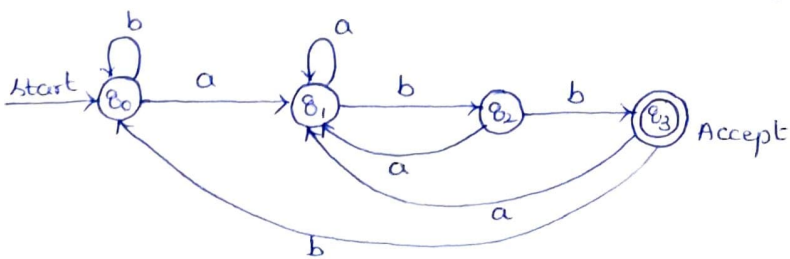
So, the language which is rejected by M1 is accepted by M2 and vice versa.

The machine M2 which accepts all those strings denoted by $\bar{L}$ that are rejected by machine M1.

So, a regular language is closed under complementation.

Example

The DFA to accept strings of a's and b's ending with abb is

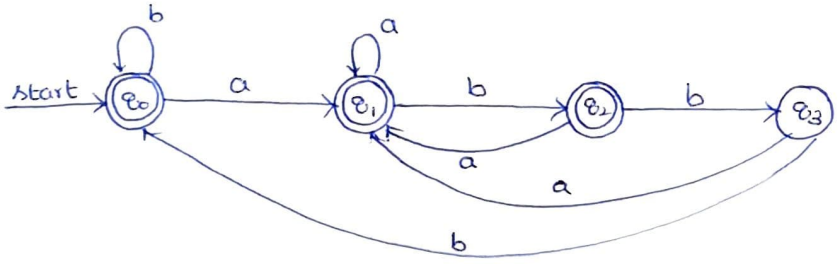


According to the theorem, the regular languages are closed under complement. So, the complement of the given language is "Strings of a's and b's that do not end with abb".

Using the closure property of complement, the DFA is obtained by

- Making the final states in above DFA as non-final states in new DFA
- Making non-final states in above DFA as final states in new DFA.

The resulting DFA accepting strings of a's and b's that do not end with abb is



CLOSURE UNDER INTERSECTION

"Show that if L and M are regular languages, then so, is $L \cap M$".

Theorem:

If L1 and L2 are regular, then the regular language is closed under intersection.

Proof:

Let $M1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ which accepts L_1 and $M2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ which accepts L_2. It is clear from these two machines that the alphabets of both machines are same. Let assume both the machines are DFA's. To accept the language $L_1 \cap L_2$, let construct the machine M that simulates both M1 and M2 where the states of the machine M are the pairs (p, q) where $p \in Q_1$, and $q \in Q_2$. The transition for the machine M from the state (p, q) on input symbol $a \in \Sigma$ is the $(\delta_1(p,a), \delta_2(q,a))$ i.e., $\delta_1(p,a) = r$ and $\delta_2(q,a) = s$, then the machine moves from the state (p, q) to the state (r, s) on input symbol a. In this manner, the machine M can simulate the effect of M1 and M2. Now, the machine $M = (Q, \Sigma, \delta, q, F)$ recognizes $L_1 \cap L_2$ where

$Q = Q_1 \times Q_2$

$q = (q_1, q_2)$ where q_1 and q_2 are the start states of machine M1 and M2 respectively.

$F = \{(p, q) \mid p \in F_1 \text{ and } q \in F_2\}$

δ: $Q \times \Sigma$ to Q is defined by $\delta((p,q), a) = (\delta_1(p,a), \delta_2(q,a))$

The string w is accepted if and only if

$\hat{\delta}((q_1, q_2), w)$ is in F

$(\hat{\delta}_1(q_1, w), \hat{\delta}_2(q_2, w))$ is in F this will happen if and only if

$\hat{\delta}_1(q_1, w) \in F_1$ and $\hat{\delta}_2(q_2, w) \in F_2$

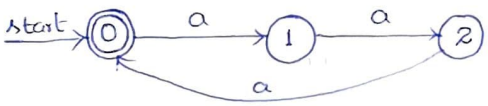
i.e., if and only if $w \in L_1 \cap L_2$.

So, the regular language is closed under intersection.

Example

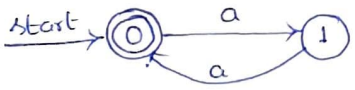
$L = \{w : |w| \bmod 3 = |w| \bmod 2$ where $w \in \Sigma^*$ and $\Sigma = \{a\}\}$

The DFA to accept a string w that results in $|w| \bmod 3$ can be written as



$Q1 = \{0, 1, 2\}$ M1

The DFA to accept a string w that results in $|w| \bmod 2$ is



$Q2 = \{0, 1\}$ M2

Transitions of DFA which has strings of w with $|w| \bmod 3 = |w| \bmod 2$ can be obtained by taking the cross product of Q1 and Q2 as,

$Q1 \times Q2 = \{(0,0), (0,1), (1,0), (1,1), (2,0), (2,1)\}$

$\delta((0,0), a) = (\delta(0,a), \delta(0,a)) = (1,1)$

$\delta((1,1), a) = (\delta(1,a), \delta(1,a)) = (2,0)$

$\delta((2,0), a) = (\delta(2,a), \delta(0,a)) = (0,1)$

$\delta((0,1), a) = (\delta(0,a), \delta(1,a)) = (1,0)$

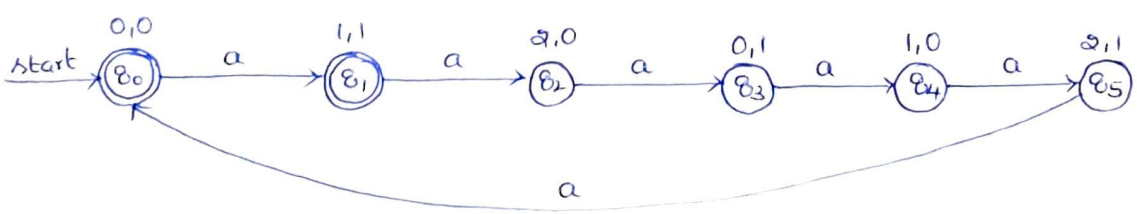
$\delta((1,0), a) = (\delta(1,a), \delta(0,a)) = (2,1)$

$\delta((2,1), a) = (\delta(2,a), \delta(1,a)) = (0,0)$

To accept strings of w such that $|w| \bmod 3 = |w| \bmod 2$, the pairs (x, y) such that $x = y$ are final states.

So, the final states are $F = \{(0,0), (1,1)\}$

So, the DFA to accept the given language is



CLOSURE UNDER DIFFERENCE

"Show that if L_1 and L_2 are regular languages, the $L_1 - L_2$ is also regular".

Theorem

If L_1 and L_2 are regular languages, then the regular language is closed under difference. (if L_1 and L_2 are regular languages, then $L_1 - L_2$ is also regular.)

Proof

Let consider $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ which accepts L_1 and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ which accepts L_2.

Then, $M = (Q, \Sigma, \delta, q, F)$ recognizing $L_1 - L_2$ is,

- Q is $Q_1 \times Q_2$ where (p, q) is in Q
- Σ is same for both the machines
- $\delta : Q \times \Sigma$ to Q is defined by $\delta((p,q), a) = (\delta_1(p,a), \delta_2(q,a))$
- $q = (q_1, q_2)$ is the start state where q_1 is the start state of M_1 and q_2 is the start state of M_2.
- $F = \{(p,q) \mid p \in F_1 \text{ and } q \notin F_2\}$

The string ω is accepted if and only if

$$\hat{\delta}((q_1, q_2), \omega) \text{ is in } F$$

i.e., $(\hat{\delta}_1(q_1, \omega), \hat{\delta}_2(q_2, \omega))$ is in F

This will happen if and only if

$$\hat{\delta}_1(q_1, \omega) \in F_1 \text{ and } \hat{\delta}_2(q_2, \omega) \notin F_2$$

i.e., if and only if ω is in $L_1 - L_2$. So, the regular language is closed under difference.

CLOSURE UNDER REVERSAL

Let $\omega = a_1 a_2 a_3 \cdots a_n$. The reversal of a string ω denoted by ω^R is defined as a string which is written backwards i.e., $\omega^R = a_n a_{n-1} \cdots a_3 a_2 a_1$.

Example : Reversal of 0111 denoted by 0111^R is 1110

Reversal of ε denoted by ε^R is ε

The Reversal of a language L is denoted by L^R. The reversal of a language L^R is defined as the set of all strings of L that are reversed.

Example : if $L = \{\varepsilon, 0, 10, 110, 11100\}$

then $L^R = \{\varepsilon, 0, 01, 011, 00111\}$

"Show that if the Language L is regular, then L^R is also regular":

Theorem

If L is regular, then L^R is also regular.

Proof

Let L is the Language corresponding to regular expression E. It is required to prove that these is another regular expression E^R such that

$$L(E) = L(E^R)$$

i.e., Language of E^R is the reversal of language of E.

Basis:

By definition of regular expression E,

- ϕ is a regular expression
- ε is a regular expression
- a is a regular expression

So, the reversal of regular expression E^R is given by,

- $\{\varepsilon\}^R = \{\varepsilon\}$
- $\{\phi\}^R = \phi$
- $\{a\}^R = \{a\}$

Induction:

By definition of regular expressions, if E_1 and E_2 are regular expressions, then:

- $E_1 + E_2$ is a regular expression
- $E_1 \cdot E_2$ is a regular expression
- E_1^* is a regular expression

Case 1: $E = E_1 + E_2$

If $E = E_1 + E_2$ is a regular expression, then $E^R = E_1^R + E_2^R$ is a RE denoting the languages: $L(E^R) = L(E_1^R) \cup L(E_2^R)$

Case 2: $E = E_1 \cdot E_2$

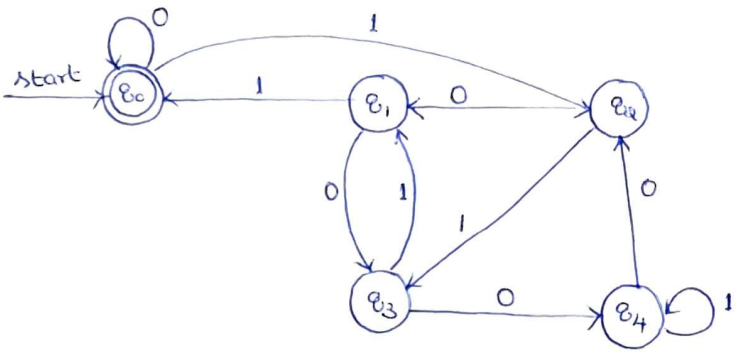
If $E = E_1 \cdot E_2$ is a RE, then $E^R = E_2^R \cdot E_1^R$ is a RE denoting the languages: $L(E^R) = L(E_2^R) \cdot L(E_1^R)$

Case 3: $E = E_1^*$

If $E = E_1^*$ is a RE, then $E^R = E_1^{*R}$ is a RE denoting the Languages $L(E^R) = L(E_1^R)$

Example

A DFA to accept the set of all strings that when interpreted in reverse as a binary integer, is divisible by 5 is,



CLOSURE UNDER HOMOMORPHISM

Let Σ and Γ are set of alphabets.

The homomorphic function $h : \Sigma \rightarrow \Gamma^*$ is called homomorphism i.e., a substitution where a single letter is replaced by a string.

If $\omega = a_1 a_2 a_3 \ldots a_n$ then $h(\omega) = h(a_1)\,h(a_2)\,h(a_3) \ldots h(a_n)$

If L is made of alphabets from Σ, then $h(L) = \{ h(\omega) \mid \omega \in L \}$ is called homomorphic image.

Example 1 : Let $\Sigma = \{0,1\}$ $\Gamma = \{0,1,2\}$ and $h(0) = 01$, $h(1) = 112$. What is $h(010)$? If $L = \{00, 010\}$, what is homomorphic image of L?

$h(\omega) = h(a_1)\,h(a_2) \ldots h(a_n)$

$h(010) = h(0)\,h(1)\,h(0)$

$= 01112101$

$L = \{00, 010\}$

$L(00,010) = L(h(00), h(010))$

$= L(h(0)h(0), h(0)h(1)h(0))$

$= L(0101, 01112101)$

Example 2 : If $\Sigma = \{0,1\}$ $\Gamma = \{1,2,3\}$, $h(0) = 3122$, $h(1) = 132$, what is $(0+1)^*(00)^*$?

$(0+1)^*(00)^* = (h(0) + h(1))^* (h(0)h(0))^*$

$= (3122 + 132)^* (31223122)^*$

Prove that if the language L is regular over an alphabet Σ, and h is a homomorphism on Σ then $h(L)$ is regular.

Theorem

If L is regular and h is homomorphism, then homomorphic image $h(L)$ is regular.

Proof

Let R be the RE and L(R) be the corresponding regular Language. We can find easily h(R) by substituting h(a) for each a in Σ. By definition of RE, h(R) is a RE and so h(L) is regular language. So, the regular language is closed under homomorphism.

LIMITATIONS OF FINITE AUTOMATON

The limitations of finite automaton are:

- An FA has finite number of states and so it does not have the capacity to remember arbitrary long amount of information.
- Since it does not have memory, FA can not remember a long string. for example; to check for matching parentheses, check whether the string is a palindrome or not etc. are not possible using FA.
- finite automata or finite state machines have trouble recognizing various types of languages involving counting, calculating, storing the string.

EQUIVALENCE AND MINIMIZATION OF FINITE AUTOMATA

Equivalence of Two States

Two states p and q of a DFA are equivalent (indistinguishable) if and only if $\delta(p,w)$ and $\delta(q,w)$ are final states or both $\delta(p,w)$ and $\delta(q,w)$ are non-final states for all $w \in \Sigma^*$.

i.e., if $\delta(q,w) \in F$ and $\delta(p,w) \in F$
then the states p and q are indistinguishable.

If $\delta(p,w) \notin F$ and $\delta(q,w) \notin F$
then also the states p and q are indistinguishable.

If there is at least one string w such that one of $\delta(p,w)$ and $\delta(q,w)$ is final state and the other is non-final state, then the states p and q are not equivalent and are called distinguishable states.

Table Filling Algorithm / Mark Procedure

The table filling algorithm is used to find the set of states that are distinguishable and indistinguishable states.

STEP 1: Identify the initial markings

For each pair (p, q) where $p \in Q$ and $q \in Q$, if $p \in F$ and $q \notin F$ or vice versa then, the pair (p, q) is distinguishable and mark the pair (p, q).

STEP 2: Identify the subsequent markings

For each pair (p, q) and for each $a \in \Sigma$, find $\delta(p, a) = r$ and $\delta(q, a) = s$. If the pair (r, s) is already marked as distinguishable then the pair (p, q) is also distinguishable and mark it as say 'x'.

Repeat step 2 until no previously unmarked pairs are marked.

Minimization of DFA (Algorithm or Procedure)

STEP 1: Find the distinguishable and indistinguishable pairs: using the table-filling algorithm.

STEP 2: Obtain the states of minimized DFA.

These groups consist of indistinguishable pairs obtained in previous step and individual distinguishable states. The groups obtained are the states of minimized DFA.

STEP 3: Compute the transition table.

If $[p_1, p_2, \cdots, p_k]$ is a group and if $\delta([p_1, p_2, \cdots, p_k], a) = [r_1, r_2, \cdots, r_m]$ then place an edge from the group $[p_1, p_2, \cdots, p_k]$ to the group $[r_1, r_2, \cdots, r_m]$ and label the edge with the symbol a. Follow this procedure for each group obtained in step 2 and for each $a \in \Sigma$.

STEP 4: Identify the start state.

If one of the component in the group $[p_1, p_2, \cdots, p_k]$ consists of a start state of given DFA then $[p_1, p_2, \cdots, p_k]$ is the start state of minimized DFA.

STEP 5: Identify the final state.

If the group $[p_1, p_2, \cdots, p_k]$ contains a final state of given DFA then the group $[p_1, p_2, \cdots, p_k]$ is final state of minimized DFA.

Q1. Obtain the distinguishable table for the automaton and then minimize the states of following DFA.

δ	a	b
$\rightarrow$ A	B	F
B	G	C
* C	A	C
D	C	G
E	H	F
F	C	G
G	G	E
H	G	C

STEP 1: Table filling algorithm

Horizontal →

B							
C	X	X					
D			X				
E			X				
F			X				
G			X				
H			X				
	A	B	C	D	E	F	G

* ↑ vertical

Initial horizontal markings : (A,C) (B,C)

Initial vertical markings : (H,C) (G,C) (F,C) (E,C) (D,C)

STEP 2:

S	a	b
✓(A,B)	(B,G)	(F,C)✓
✓(A,D)	(B,C)✓	(F,G)
(A,E)	(B,H)	(F,F)
✓(A,F)	(B,C)✓	(F,G)
(A,G)	(B,G)	(F,E)
✓(A,H)	(B,G)	(F,C)✓
✓(B,D)	(G,C)✓	(C,G)
✓(B,E)	(G,H)	(C,F)•
✓(B,F)	(G,C)✓	(C,G)
✓(B,G)	(G,G)	(C,E)•
(B,H)	(G,G)	(C,C)
✓(D,E)	(C,H)•	(G,F)
(D,F)	(C,C)	(G,G)
✓(D,G)	(C,G)•	(G,E)
✓(D,H)	(C,G)	(G,C)✓
✓(E,F)	(H,C)✓	(F,G)
(E,G)	(H,G)	(F,E)
✓(E,H)	(H,G)	(F,C)✓
✓(F,G)	(C,G)•	(G,E)
✓(F,H)	(C,G)	(G,C)✓
✓(G,H)	(G,G)	(E,C)✓

B	X						
C	X	X					
D	X	X	X				
E		X	X	X			
F	X	X	X		X		
G		X	X	X		X	
H	X		X	X	X	X	X
	A	B	C	D	E	F	G

*

S	a	b
(A,E)	(B,H)	(F,F)
✓(A,G)	(B,G)✓	(F,E)
(B,H)	(G,G)	(C,C)
(D,F)	(C,C)	(G,G)
✓(E,G)	(H,G)•	(F,E)

B	X						
C	X	X					
D	X	X	X				
E		X	X	X			
F	X	X	X		X		
G	X	X	X	X	X	X	
H	X		X	X	X	X	X
	A	B	C	D	E	F	G

*

Minimizing DFA

STEP 1 : (A,E) (B,H) and (D,F) are indistinguishable

C, G are distinguishable

STEP 2 : The states of the minimized DFA

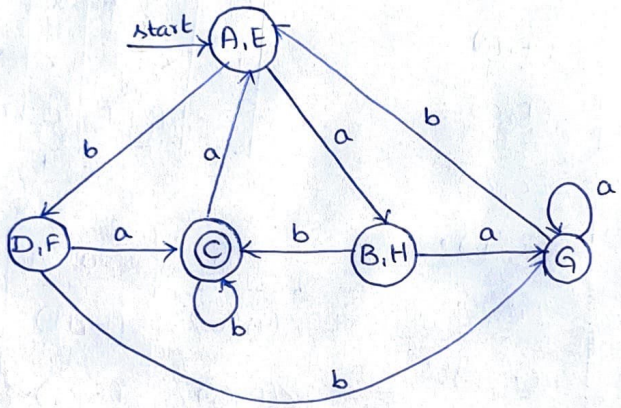
(A,E), (B,H), C, (D,F) and G

STEP 3 :

	S	a	b
→	(A,E)	(B,H)	(D,F) (F,F)
	(B,H)	(G,G) G	(C,C) C
*	C	(A,E) A	C
	(D,F)	C	G
	G	G	(A,E) E

STEP 4 : (A,E) is the start state of minimized DFA.

STEP 5 : C is the final state state of minimized DFA.



Q2: Find the minimized DFA for the following

	S	0	1
→	A	B	A
	B	A	C
	C	D	B
*	D	D	A
	E	D	F
	F	G	E
	G	F	G
	H	G	D

STEP 1 :

B								
C								
D	x	x	x					
E				x				
F				x				
G				x				
H				x				
	A	B	C	D	E	F	G	

→ * Horizontal

* ↑ Vertical

Table filling algorithm

Horizonal markings

(A,D) (B,D) (C,D)

Vertical markings

(D,E) (D,F) (D,G) (D,H)

STEP 2 :

S	a	b
(A,B)	(B,A)	(A,C)
✓(A,C)	(B,D)✓	(A,B)
✓(A,E)	(B,D)✓	(A,F)
(A,F)	(B,G)	(A,E)
(A,G)	(B,F)	(A,G)
✓(A,H)	(B,G)	(A,D)✓
✓(B,C)	(A,D)✓	(C,B)
✓(B,E)	(A,D)✓	(C,F)
(B,F)	(A,G)	(C,E)
(B,G)	(A,F)	(C,G)
✓(B,H)	(A,G)	(C,D)✓
(C,E)	(D,D)	(B,F)
✓(C,F)	(D,G)✓	(B,E)
✓(C,G)	(D,F)✓	(B,G)
✓(C,H)	(D,G)	(B,D)✓
✓(E,F)	(D,G)✓	(F,E)
✓(E,G)	(D,F)✓	(F,G)
✓(E,H)	(D,G)✓	(F,D)
(F,G)	(G,F)	(E,G)
✓(F,H)	(G,G)	(E,D)✓
✓(G,H)	(F,G)	(G,D)✓

B								
C	x	x						
D	x	x	x					
E	x	x		x				
F			x	x	x			
G			x	x	x			
H	x	x	x	x	x	x	x	
	A	B	C	D	E	F	G	

* *

S	0	1
✓(A,B)	(B,A)	(A,C)✓
✓(A,F)	(B,G)	(A,E)✓
(A,G)	(B,F)	(A,G)
(B,F)	(A,G)	(C,E)
✓(B,G)	(A,F)	(C,G)✓
(C,E)	(D,D)	(B,F)
✓(F,G)	(G,F)	(E,G)✓

B	x							
C	x	x						
D	x	x	x					
E	x	x		x				
F	x		x	x	x			
G		x	x	x	x	x		
H	x	x	x	x	x	x	x	
	A	B	C	D	E	F	G	

* *

Minimizing DFA

STEP 1 : (A,G), (B,F), and (C,E) are indistinguishable.
D and H are distinguishable.

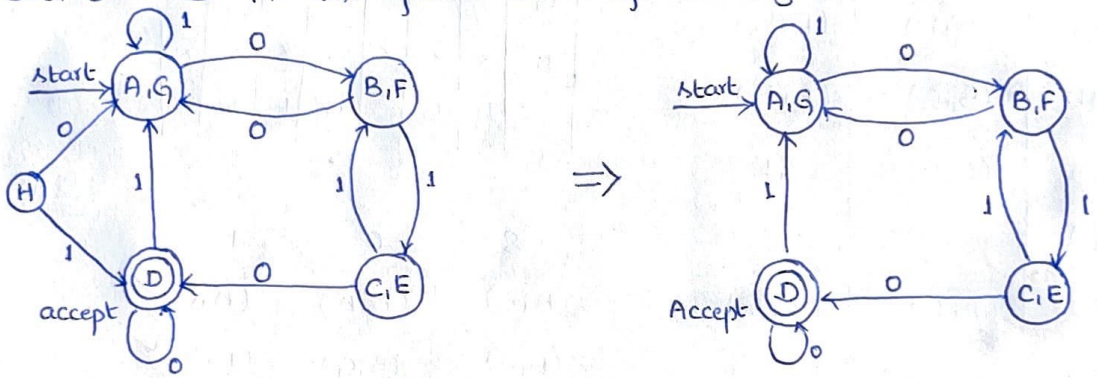
STEP 2 : The states of the minimized DFA
(A,G), (B,F), (C,E), D and H

STEP 3 :

δ	0	1
→ (A,G)	(B,F)	(A,G)
(B,F)	(A,G)	(C,E)
(C,E)	(D,D)	(B,F)
* D	D	(A,G) A
H	(A,G) G	D

STEP 4 : (A,G) is the start state of minimized DFA.

STEP 5 : D is the final state of minimized DFA.



Q3. Minimize the following DFA using table-filling algorithm where A is the start state. The states C, F and I are final states.

δ	0	1
→ A	B	E
B	C	F
* C	D	H
D	E	H
E	F	I
* F	G	B
G	H	B
H	I	C
* I	A	E

STEP 1:

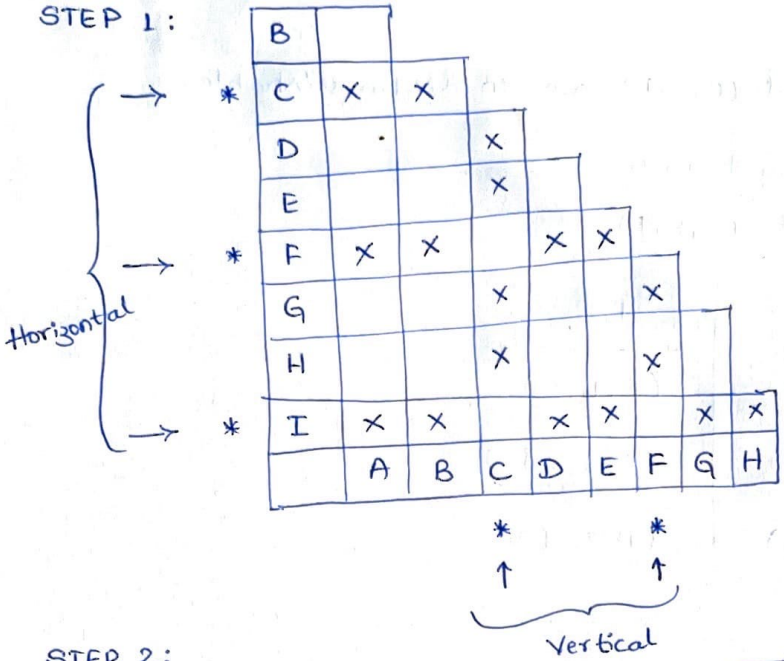


Table filling algorithm

Horizontal markings

(A,C) (B,C)
(A,F) (B,F) (D,F) (E,F)
(A,I) (B,I) (D,I) (E,I) (G,I) (H,I)

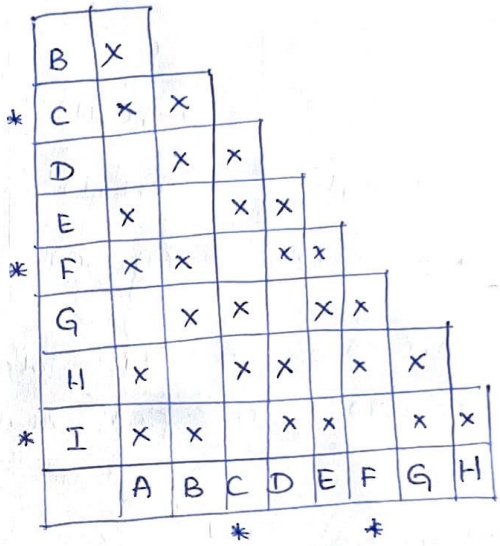
Vertical markings

(D,C) (E,C) (G,C) (H,C)
(G,F) (H,F)

(C,F) ×
(C,I) ×
(F,I) × → both are final states

STEP 2:

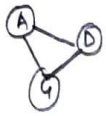
S	0	1
✓ (A,B)	(B, C) ✓	(E, F)
(A,D)	(B, E)	(E, H)
✓ (A,E)	(B, F) ✓	(E, I)
(A,G)	(B, H)	(E, B)
✓ (A,H)	(B, I) ✓	(E, C)
✓ (B,D)	(C, E) ✓	(F, H)
(B,E)	(C, F)	(F, I)
✓ (B,G)	(C, H)	(F, B) ✓
(B,H)	(C, I)	(F, C)
(C,F)	(D,G)	(H,B)
(C,I)	(D,A)	(H,E)
✓ (D,E)	(E, F) ✓	(H, I)
(D,G)	(E, H)	(H, B)
✓ (D,H)	(E, I) ✓	(H, C)
✓ (E,G)	(F, H)	(I, B) ✓
(E,H)	(F, I)	(I, C)
(F,I)	(G, A)	(B, E)
✓ (G,H)	(H, I)	(B, C) ✓



S	0	1
(A,D)	(B,E)	(E,H)
(A,G)	(B,H)	(E,B)
(B,E)	(C,F)	(F,I)
(B,H)	(C,I)	(F,C)
(D,G)	(E,H)	(H,B)
(F,I)	(G,A)	(B,E)
(E,H)	(F,I)	(I,C)
(C,F)	(D,G)	(H,B)
(C,I)	(D,A)	(H,E)

Indistinguishable pairs

(A,D) (A,G) (D,G) = (A,D,G)
(B,E) (B,H) (E,H) = (B,E,H)
(C,F) (C,I) (F,I) = (C,F,I)



Minimizing DFA

STEP 1 : (A,D,G) , (B,E,H) and (C,F,I) are in-distinguishable.

STEP 2 : The states of minimized DFA
(A,D,G) , (B,E,H) and (C,F,I)

STEP 3 :

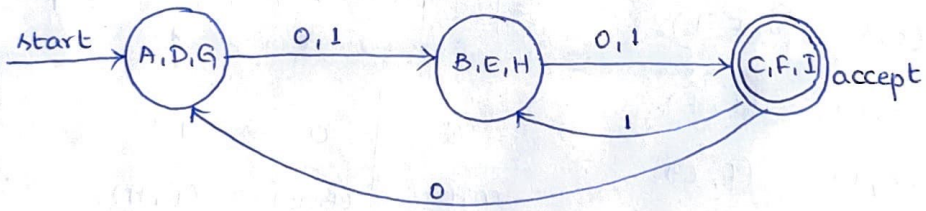
S	0	1
→ (A,D,G)	(B,E,H)	(E,H,B)
(B,E,H)	(C,F,I)	(F,I,C)
* (C,F,I)	(D,G,A)	(H,B,E)

⇓

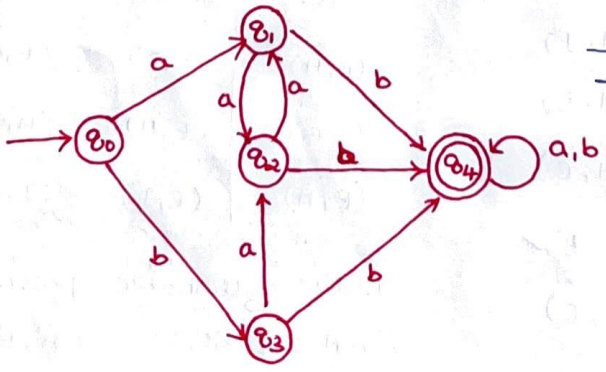
S	0	1
→ (A,D,G)	(B,E,H)	(B,E,H)
(B,E,H)	(C,F,I)	(C,F,I)
* (C,F,I)	(A,D,G)	(B,E,H)

STEP 4 : (A,D,G) is the start state of minimized DFA.

STEP 5 : (C,F,I) is the final state of minimized DFA.



Q4. Minimize the following DFA.



S	a	b
→ q_0	q_1	q_3
q_1	q_2	q_4
q_2	q_1	q_4
q_3	q_2	q_4
* q_4	q_4	q_4

Table filling algorithm

STEP 1 :

	q_1				
	q_2				
	q_3				
*	q_4	x	x	x	x
		q_0	q_1	q_2	q_3

STEP 2 :

	δ	a	b
✓	(q_0, q_1)	(q_1, q_2)	(q_3, q_4) ✓
✓	(q_0, q_2)	(q_1, q_1)	(q_3, q_4) ✓
✓	(q_0, q_3)	(q_1, q_2)	(q_3, q_4) ✓
	(q_1, q_2)	(q_2, q_1)	(q_4, q_4)
	(q_1, q_3)	(q_2, q_2)	(q_4, q_4)
	(q_2, q_3)	(q_1, q_2)	(q_4, q_4)

	q_1	x			
	q_2	x			
	q_3	x			
*	q_4	x	x	x	x
		q_0	q_1	q_2	q_3

δ	a	b
(q_1, q_2)	(q_2, q_1)	q_4
(q_1, q_3)	q_2	q_4
(q_2, q_3)	(q_1, q_2)	q_4

Indistinguishable pairs

(q_1, q_2) (q_1, q_3) = (q_1, q_2, q_3)

(q_2, q_3)

Minimizing DFA

STEP 1 : (q_1, q_2, q_3) is the in-distinguishable.

q_0 and q_4 are the distinguishable.

STEP 2 : The states of minimized DFA - q_0, (q_1, q_2, q_3), and q_4

STEP 3 :

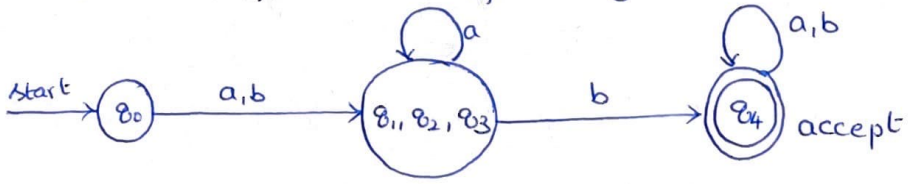
	δ	a	b
→	q_0	q_1	q_3
	(q_1, q_2, q_3)	(q_1, q_2)	q_4
*	q_4	q_4	q_4

⇒

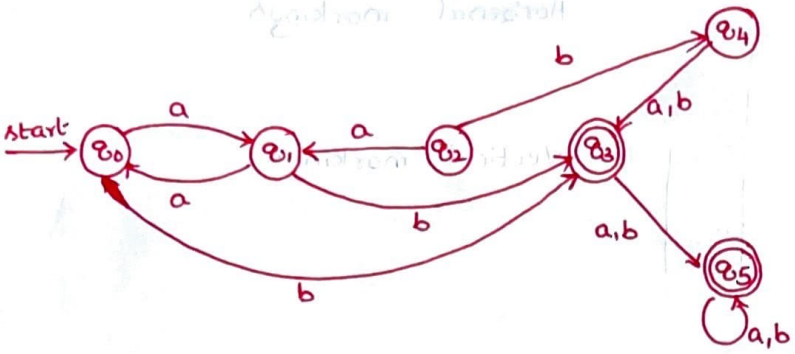
	δ	a	b
→	q_0	(q_1, q_2, q_3)	(q_1, q_2, q_3)
	(q_1, q_2, q_3)	(q_1, q_2, q_3)	q_4
*	q_4	q_4	q_4

STEP 4 : q_0 is the start state of minimized DFA.

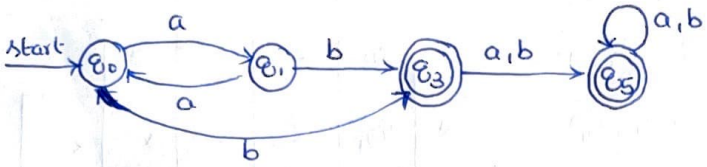
STEP 5 : q_4 is the final state of minimized DFA.



Q5. Minimize the following DFA



It is clear from the above DFA that q_2 and q_4 are not reachable from the start state, So that they can be removed.



STEP 1:

	q_1		
*	q_3	x	x
*	q_5	x	x
		q_0	q_1	q_3

*

STEP 2:

δ	a	b
(q_0, q_1)	q_1, q_0	q_3, q_3
(q_3, q_5)	q_5	q_5

Indistinguishable pairs

(q_0, q_1) and (q_3, q_5)

Minimizing DFA

STEP 1: (q_0, q_1) and (q_3, q_5) are indistinguishable.

STEP 2: The states of minimized DFA - (q_0, q_1) and (q_3, q_5)

STEP 3:

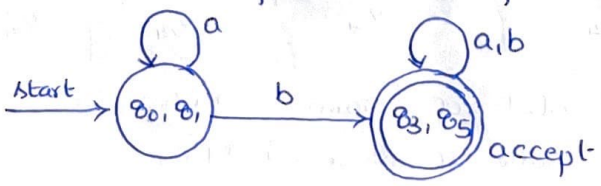
	δ	a	b
→	(q_0, q_1)	q_1	q_3
*	(q_3, q_5)	(q_3, q_5)	$\{q_3, q_5\}$

⇒

	δ	a	b
→	(q_0, q_1)	(q_0, q_1)	(q_3, q_5)
*	(q_3, q_5)	(q_3, q_5)	(q_3, q_5)

STEP 4: (q_0, q_1) is the start state of minimized DFA.

STEP 5: (q_3, q_5) is the final state of minimized DFA.



Q6. Construct a minimum automation equivalent to given automation 'M' whose transition table given below:

δ	0	1
$\rightarrow q_0$	q_0	q_3
q_1	q_2	q_5
q_2	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
$*q_6$	q_1	q_3

Table filling algorithm

STEP 1 :

q_1						
q_2						
q_3						
q_4						
q_5						
$*q_6$	x	x	x	x	x	x
	q_0	q_1	q_2	q_3	q_4	q_5

q_1						
q_2						
q_3						
q_4	x	x	x	x		
q_5					x	
$*q_6$	x	x	x	x	x	x
	q_0	q_1	q_2	q_3	q_4	q_5

STEP 2 :

δ	0	1
(q_0, q_1)	(q_0, q_2)	(q_3, q_5)
(q_0, q_2)	(q_0, q_3)	(q_3, q_4)
(q_0, q_3)	(q_0, q_0)	(q_3, q_5)
✓ (q_0, q_4)	(q_0, q_0)	(q_3, q_6) ✓
(q_0, q_5)	(q_0, q_1)	(q_3, q_4)
(q_1, q_2)	(q_2, q_3)	(q_5, q_4)
(q_1, q_3)	(q_2, q_0)	(q_5, q_5)
✓ (q_1, q_4)	(q_2, q_0)	(q_5, q_6) ✓
(q_1, q_5)	(q_2, q_1)	(q_5, q_4)
(q_2, q_3)	(q_3, q_0)	(q_4, q_5)
✓ (q_2, q_4)	(q_3, q_0)	(q_4, q_6) ✓
(q_2, q_5)	(q_3, q_1)	(q_4, q_4)
✓ (q_3, q_4)	(q_0, q_0)	(q_5, q_6) ✓
(q_3, q_5)	(q_0, q_1)	(q_5, q_4)
✓ (q_4, q_5)	(q_0, q_1)	(q_6, q_4) ✓

δ	0	1
(q_0, q_1)	(q_0, q_2)	(q_3, q_5)
✓ (q_0, q_2)	(q_0, q_3)	(q_3, q_4) ✓
(q_0, q_3)	(q_0, q_0)	(q_3, q_5)
✓ (q_0, q_5)	(q_0, q_1)	(q_3, q_4) ✓
✓ (q_1, q_2)	(q_2, q_3)	(q_5, q_4) ✓
(q_1, q_3)	(q_2, q_0)	(q_5, q_5)
✓ (q_1, q_5)	(q_2, q_1)	(q_5, q_4) ✓
✓ (q_2, q_3)	(q_3, q_0)	(q_4, q_5) ✓
(q_2, q_5)	(q_3, q_1)	(q_4, q_4)
✓ (q_3, q_5)	(q_0, q_1)	(q_5, q_4) ✓

q_1						
q_2	X	X				
q_3			X			
q_4	X	X	X	X		
q_5	X	X		X	X	
* q_6	X	X	X	X	X	X
	q_0	q_1	q_2	q_3	q_4	q_5

S	0	1
✓ (q_0, q_1)	(q_0, q_2)	(q_3, q_5) ✓
✓ (q_0, q_3)	(q_0, q_0)	(q_3, q_5) ✓
✓ (q_1, q_3)	(q_2, q_0) ✓	(q_5, q_5)
(q_2, q_5)	(q_3, q_1)	(q_4, q_4)

q_1	X					
q_2	X	X				
q_3	X	X	X			
q_4	X	X	X	X		
q_5	X	X		X	X	
* q_6	X	X	X	X	X	X
	q_0	q_1	q_2	q_3	q_4	q_5

S	0	1
✓ (q_2, q_5)	(q_3, q_1) ✓	(q_4, q_4)

q_1	X					
q_2	X	X				
q_3	X	X	X			
q_4	X	X	X	X		
q_5	X	X	X	X	X	
* q_6	X	X	X	X	X	X
	q_0	q_1	q_2	q_3	q_4	q_5

Minimizing DFA.

STEP 1: No in-distinguishable states.

$q_0, q_1, q_2, q_3, q_4, q_5, q_6$ are distinguishable.

STEP 2: The states of the minimized DFA are

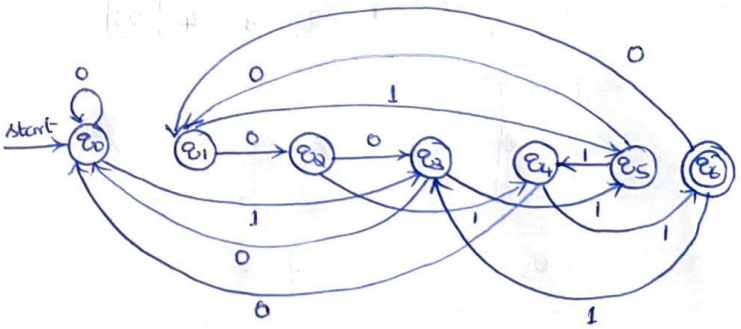
$q_0, q_1, q_2, q_3, q_4, q_5$ & q_6

STEP 3:

S	0	1
→ q_0	q_0	q_3
q_1	q_2	q_5
q_2	q_3	q_4
q_3	q_0	q_5
q_4	q_0	q_6
q_5	q_1	q_4
* q_6	q_1	q_3

STEP 4: q_0 is the start state of the minimized DFA.

STEP 5: q_6 is the final state of the minimized DFA.



No minimized DFA for the given automation 'M'.

Q7. Consider the DFA given by the transition table

δ	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_1	q_3
q_2	q_1	q_2
q_3	q_1	q_4
$*\ q_4$	q_1	q_2

i. Draw the table of distinguishabilities for this automaton.

ii. Construct the minimum state equivalent DFA.

iii. Write the language accepted by the DFA.

(i) Table filling algorithm

STEP 1 :

q_1				
q_2				
q_3				
$*\ q_4$	x	x	x	x
	q_0	q_1	q_2	q_3

STEP 2 :

δ	a	b
(q_0, q_1)	(q_1, q_1)	(q_2, q_3)
(q_0, q_2)	(q_1, q_1)	(q_2, q_2)
✓ (q_0, q_3)	(q_1, q_1)	(q_2, q_4) ✓
(q_1, q_2)	(q_1, q_1)	(q_3, q_2)
✓ (q_1, q_3)	(q_1, q_1)	(q_3, q_4) ✓
✓ (q_2, q_3)	(q_1, q_1)	(q_2, q_4) ✓

q_1				
q_2				
q_3	x	x	x	
$*\ q_4$	x	x	x	x
	q_0	q_1	q_2	q_3

δ	a	b
✓ (q_0, q_1)	(q_1, q_1)	(q_2, q_3) ✓
(q_0, q_2)	(q_1, q_1)	(q_2, q_2)
✓ (q_1, q_2)	(q_1, q_1)	(q_3, q_2) ✓

q_1	x			
q_2		x		
q_3	x	x	x	
$*\ q_4$	x	x	x	x
	q_0	q_1	q_2	q_3

(ii) Minimizing DFA

STEP 1 : (q_0, q_2) is in-distinguishable.

q_1, q_3, q_4 are distinguishable.

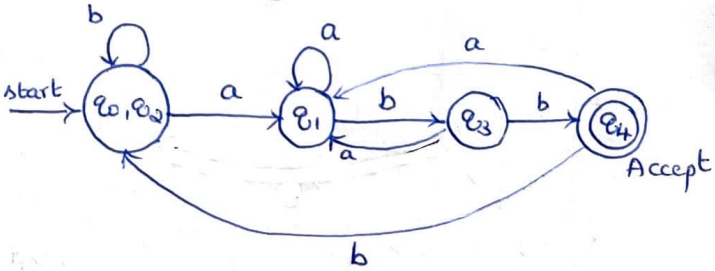
STEP 2 : (q_0, q_2), q_1, q_3, q_4 are the states of the minimized DFA.

STEP 3 :

δ	a	b
$\rightarrow (q_0, q_2)$	q_1	q_2
q_1	q_1	q_3
q_3	q_1	q_4
$*\ q_4$	q_1	(q_0, q_2)

STEP 4 : (q_0, q_2) is the start state of the minimized DFA.

STEP 5 : q_4 is the final state of the minimized DFA.



(iii) The language accepted by the DFA is "Strings of a's and b's ends with abb".

Q8. When two states are equivalent or distinguishable ? Minimize the following DFA using table filling algorithm.

δ	0	1
$\rightarrow q_1$	q_2	q_3
q_2	q_3	q_5
$*\ q_3$	q_4	q_3
q_4	q_3	q_5
$*\ q_5$	q_2	q_5

Table filling algorithm
STEP 1 :

q_2				
$*\ q_3$	x	x		
q_4			x	
$*\ q_5$	x	x		x
	q_1	q_2	q_3	q_4

$*$

δ	0	1
✓(q_1, q_2)	(q_2, q_3) ✓	(q_3, q_5)
✓(q_1, q_4)	(q_2, q_3) ✓	(q_3, q_5)
(q_2, q_4)	(q_3, q_3)	(q_5, q_5)
(q_3, q_5)	(q_4, q_2)	(q_3, q_5)

q_2	x			
$*\ q_3$	x	x		
q_4	x		x	
$*\ q_5$	x	x		x
	q_1	q_2	q_3	q_4

If there is at least one string w such that one of $\delta(p,w)$ and $\delta(q,w)$ is final state and the other is non-final state, then the states p and q are not equivalent and are called distinguishable states.

Two states p and q of a DFA are equivalent (indistinguishable) if and only if $\delta(p,w)$ and $\delta(q,w)$ are final states or both $\delta(p,w)$ and $\delta(q,w)$ are non-final states for all $w \in \Sigma^*$.

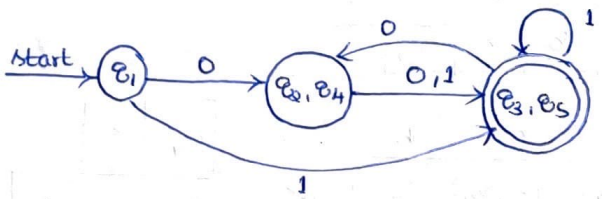
Minimizing DFA

STEP 1 : (q_2, q_4), (q_3, q_5) are in-distinguishable.
q_1 is distinguishable.

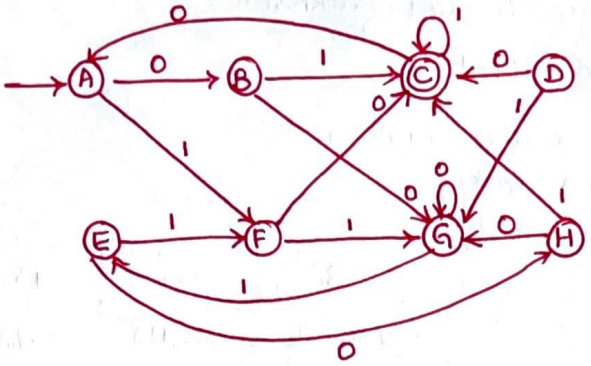
STEP 2 : The states of the minimized DFA are
q_1, (q_2, q_4) and (q_3, q_5)

STEP 3 :

δ	0	1
$\rightarrow q_1$	(q_2, q_4)	(q_3, q_5)
(q_2, q_4)	(q_3, q_5)	(q_3, q_5)
$*\ (q_3, q_5)$	(q_2, q_4)	(q_3, q_5)



Q9. Consider the DFA given by the transition diagram



i. Draw the table of distinguishabilities for this automaton

ii. Construct the minimum state equivalent DFA.

STEP 1 : Table filling algorithm

B							
* C	X	X					
D			X				
E			X				
F			X				
G			X				
H			X				
	A	B	C	D	E	F	G

*

S	0	1
✓ (A, B)	(B, G)	(F, C) ✓
✓ (F, G)	(C, G) ✓	(G, E)
✓ (A, D)	(B, C) ✓	(F, G)
✓ (G, H)	(G, G)	(E, C) ✓
(A, E)	(B, H)	(F, F)
✓ (A, F)	(B, C) ✓	(F, G)
(A, G)	(B, G)	(F, E)
✓ (A, H)	(B, G)	(F, C) ✓
✓ (B, D)	(G, C) ✓	(C, G)
✓ (B, E)	(G, H)	(C, F) ✓
✓ (B, F)	(G, C)	(C, G) ✓
✓ (B, G)	(G, G)	(C, E) ✓
(B, H)	(G, G)	(C, C)
✓ (D, E)	(C, H) ✓	(G, F)
(D, F)	(C, C)	(G, G)
✓ (D, G)	(C, G) ✓	(G, E)
✓ (D, H)	(C, G) ✓	(G, C)
✓ (E, F)	(H, C) ✓	(F, G)
(E, G)	(H, G)	(F, E)
✓ (E, H)	(H, G)	(F, C) ✓
✓ (F, H)	(C, G) ✓	(G, E)

S	0	1
→ A	B	F
B	G	C
* C	A	C
D	C	G
E	H	F
F	C	G
G	G	E
H	G	C

B	X						
* C	X	X					
D	X	X	X				
E		X	X	X			
F	X	X	X		X		
G	X	X	X	X	X	X	
H	X		X	X	X	X	X
	A	B	C	D	E	F	G

*

S	0	1
(A, E)	(B, H)	F
✓ (A, G)	(B, G) ✓	(F, E)
(B, H)	G	C
(D, F)	C	G
✓ (E, G)	(H, G) ✓	(F, E)

Minimizing DFA.

STEP 1 : (A,E), (B,H), (D,F) are in-distinguishable.
C and G are distinguishable:

STEP 2 : The states of the minimized DFA are (A,E), (B,H), C, (D,F) and G.

STEP 3 :

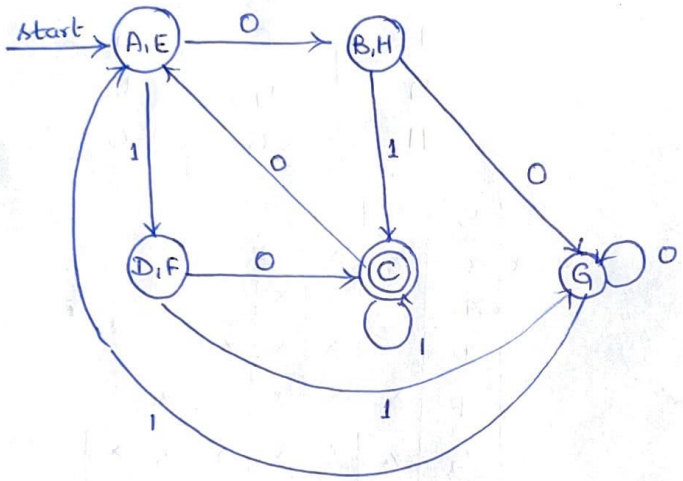
S	0	1
→ (A,E)	(B,H)	(F,F)
(B,H)	(G,G)	(C,C)
* C	A	C
(D,F)	(C,C)	(G,G)
G	G	E

⟹

S	0	1
→ (A,E)	(B,H)	(D,F)
(B,H)	G	C
* C	(A,E)	C
(D,F)	C	G
G	G	(A,E)

STEP 4 : (A,E) is the start state of the minimized DFA.

STEP 5 : C is the final state of the minimized DFA.



Q10. Construct the product of DFA.



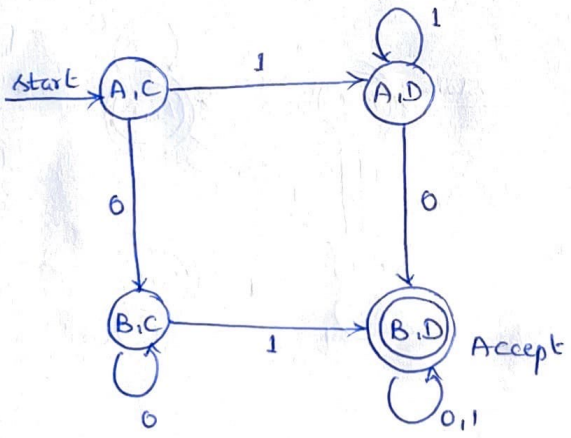
S	0	1
→ A	B	A
* B	B	B

S	0	1
→ C	C	D
* D	D	D

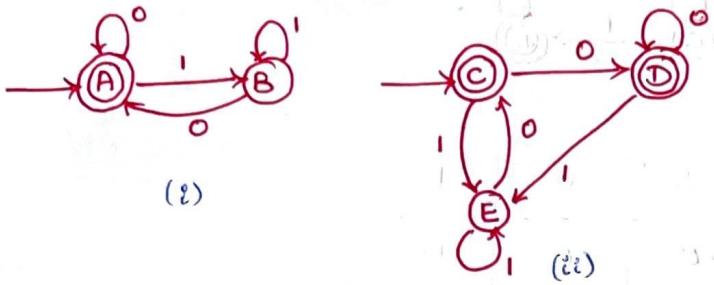
$Q1 = \{A, B\}$ $Q2 = \{C, D\}$

$Q1 \times Q2 = \{(A,C), (A,D), (B,C), (B,D)\}$

S	0	1
→ (A,C)	(B,C)	(A,D)
(A,D)	(B,D)	(A,D)
(B,C)	(B,C)	(B,D)
* (B,D)	(B,D)	(B,D)



Q11. Consider the two DFA's shown below. Using table filling algorithm, show that the language accepted by both the DFA's is same.



(i) (ii)

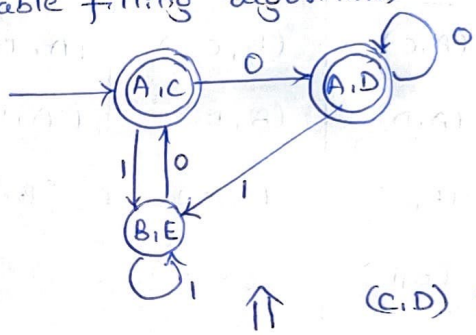
(i) The language accepted by first DFA is
"Strings of 0's and 1's ending with 0 including null string."

(ii) The language accepted by second DFA is
"Strings of 0's and 1's ending with 0 including null string."

So, the RE is $\varepsilon + (0+1)^* 0$

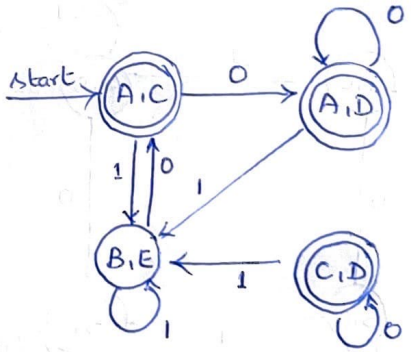
Consider all the states for table filling algorithm

	B	X		
*	C		X	
*	D		X	
	E	X		X	X
		A	B	C	D
		*		*	*



(C,D) is not reachable from start state (A,C) So, remove (C,D).

S	0	1
(A,C)	(A,D)	(B,E)
(A,D)	(A,D)	(B,E)
(B,E)	(A,C)	(B,E)
(C,D)	(D,D)	(E,E)



A,C,D
B,E

(A,C), (A,D), (B,E), (C,D) are in-distinguishable or equivalent pairs. Since, A and C are found equivalent and those states were the start states of the two original automata, we conclude that these DFA's do accept the same language.

The RE $\varepsilon + (0+1)^* 0$ will be accepted by the minimized DFA. So, both DFA's will accept the same language.